

Отчёт по лабораторной работе №5

Расширенная настройка HTTP-сервера Apache

Гашимова Эсма Эльшан кызы

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Генерация криптографического ключа и самоподписанного сертификата	6
2.2	Настройка виртуального хоста для работы по HTTPS	7
2.3	Настройка межсетевого экрана для HTTPS	10
2.4	Проверка работы веб-сервера по HTTPS	11
2.5	Настройка HTTP-сервера для работы с PHP	13
2.6	Внесение изменений в настройки внутреннего окружения виртуальной машины	16
3	Вывод	18
4	Контрольные вопросы	19

Список иллюстраций

2.1	Генерация закрытого ключа и самоподписанного сертификата .	7
2.2	Конфигурация виртуального хоста для HTTP и HTTPS	8
2.3	Разрешение HTTPS в firewalld и перезапуск HTTP-сервера	11
2.4	Предупреждение браузера при использовании самоподписанного сертификата	12
2.5	Проверка доступа к веб-серверу по протоколу HTTPS	12
2.6	Просмотр данных самоподписанного сертификата	13
2.7	Установка PHP и необходимых зависимостей	14
2.8	Создание файла index.php с вызовом phpinfo	14
2.9	Настройка прав, восстановление контекстов SELinux и перезапуск Apache	15
2.10	Проверка выполнения PHP на веб-сервере	15
2.11	Копирование конфигурации, веб-контента, ключа и сертификата в проект Vagrant	16
2.12	Скрипт автоматической настройки HTTP-сервера с поддержкой HTTPS и PHP	17

Список таблиц

1 Цель работы

Приобретение практических навыков по расширенному конфигурированию HTTP-сервера Apache в части безопасности и возможности использования PHP.

2 Выполнение лабораторной работы

2.1 Генерация криптографического ключа и самоподписанного сертификата

На виртуальной машине `server` перехожу в режим суперпользователя. Для хранения закрытого ключа создаю каталог `/etc/pki/tls/private`, после чего создаю символическую ссылку `/etc/ssl/private` на данный каталог. Затем перехожу в каталог с закрытыми ключами и с помощью `openssl` генерирую новый закрытый RSA-ключ длиной 2048 бит и самоподписанный сертификат X.509 для веб-сервера `www.eegashimova.net` (см. рис. Рисунок 2.1).

При формировании сертификата параметр `-x509` задаёт создание самоподписанного сертификата X.509, параметр `-nodes` отключает защиту закрытого ключа парольной фразой, а `-newkey rsa:2048` создаёт новый RSA-ключ длиной 2048 бит. Параметры `-keyout` и `-out` задают имена файлов закрытого ключа и сертификата соответственно.


```

www.eegashimova.net.conf [----] 11 L:[ 1+23 24/ 24] *(863 / 863b) <EOF>
<VirtualHost *:80>
    ServerAdmin webmaster@eegashimova.net
    DocumentRoot /var/www/html/www.eegashimova.net
    ServerName www.eegashimova.net
    ServerAlias www.eegashimova.net
    ErrorLog logs/www.eegashimova.net-error_log
    CustomLog logs/www.eegashimova.net-access_log common
    RewriteEngine on
    RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
</VirtualHost>

<IfModule mod_ssl.c>
<VirtualHost *:443>
    SSLEngine on
    ServerAdmin webmaster@eegashimova.net
    DocumentRoot /var/www/html/www.eegashimova.net
    ServerName www.eegashimova.net
    ServerAlias www.eegashimova.net
    ErrorLog logs/www.eegashimova.net-error_log
    CustomLog logs/www.eegashimova.net-access_log common
    SSLCertificateFile /etc/ssl/certs/www.eegashimova.net.crt
    SSLCertificateKeyFile /etc/ssl/private/www.eegashimova.net.key
</VirtualHost>
</IfModule>

```

Рисунок 2.2: Конфигурация виртуального хоста для HTTP и HTTPS

Конфигурационный файл содержит следующие параметры:

```

<VirtualHost *:80>

    ServerAdmin webmaster@eegashimova.net

    DocumentRoot /var/www/html/www.eegashimova.net

    ServerName www.eegashimova.net

    ServerAlias www.eegashimova.net

    ErrorLog logs/www.eegashimova.net-error_log

    CustomLog logs/www.eegashimova.net-access_log common

    RewriteEngine on

    RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]

</VirtualHost>

<IfModule mod_ssl.c>
<VirtualHost *:443>

    SSLEngine on

    ServerAdmin webmaster@eegashimova.net

```

```
DocumentRoot /var/www/html/www.eegashimova.net
ServerName www.eegashimova.net
ServerAlias www.eegashimova.net
ErrorLog logs/www.eegashimova.net-error_log
CustomLog logs/www.eegashimova.net-access_log common
SSLCertificateFile /etc/ssl/certs/www.eegashimova.net.crt
SSLCertificateKeyFile /etc/ssl/private/www.eegashimova.net.key
</VirtualHost>
</IfModule>
```

Строка `<VirtualHost *:80>` открывает описание виртуального хоста, принимающего соединения на стандартном HTTP-порту 80.

Директива `ServerAdmin webmaster@eegashimova.net` задаёт адрес электронной почты администратора веб-сервера.

Директива `DocumentRoot /var/www/html/www.eegashimova.net` указывает каталог, содержащий файлы веб-сайта.

Директива `ServerName www.eegashimova.net` определяет основное доменное имя данного виртуального хоста.

Директива `ServerAlias www.eegashimova.net` определяет дополнительное имя, по которому виртуальный хост может обрабатывать запросы.

Директива `ErrorLog logs/www.eegashimova.net-error_log` задаёт файл журнала, в который Apache записывает сообщения об ошибках данного виртуального хоста.

Директива `CustomLog logs/www.eegashimova.net-access_log common` задаёт отдельный журнал запросов к виртуальному хосту и использует стандартный формат журналирования `common`.

Директива `RewriteEngine on` включает механизм преобразования URL модуля `mod_rewrite`.

Директива `RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]` перена-

правляет любой запрос, поступивший по HTTP, на аналогичный адрес с использованием HTTPS. Флаг `R=301` задаёт постоянное перенаправление с кодом HTTP 301, а флаг `L` указывает, что после срабатывания данного правила дальнейшие правила преобразования применять не требуется.

Строка `</VirtualHost>` завершает описание виртуального хоста на порту 80.

Строка `<IfModule mod_ssl.c>` указывает, что расположенная внутри конфигурация используется при наличии загруженного модуля Apache `mod_ssl`.

Строка `<VirtualHost *:443>` открывает описание защищённого виртуального хоста, принимающего HTTPS-соединения на порту 443.

Директива `SSLEngine on` включает использование SSL/TLS для данного виртуального хоста.

Директивы `ServerAdmin`, `DocumentRoot`, `ServerName`, `ServerAlias`, `ErrorLog` и `CustomLog` выполняют те же функции, что и в виртуальном хосте на порту 80, но применяются к HTTPS-соединениям.

Директива `SSLCertificateFile /etc/ssl/certs/www.eegashimova.net.crt` указывает путь к файлу сертификата, который веб-сервер передаёт клиенту при установлении защищённого соединения.

Директива `SSLCertificateKeyFile /etc/ssl/private/www.eegashimova.net.key` задаёт путь к закрытому ключу, соответствующему используемому сертификату.

Строка `</VirtualHost>` завершает описание HTTPS-виртуального хоста, а `</IfModule>` завершает условный блок конфигурации модуля `mod_ssl`.

2.3 Настройка межсетевого экрана для HTTPS

Для обеспечения доступа клиентов к веб-серверу по HTTPS добавляю сервис `https` в текущую и постоянную конфигурации `firewalld`. После внесения изменений перезагружаю правила межсетевого экрана и перезапускаю служ-

бу httpd. Команды завершаются успешно, следовательно, сервер готов принимать защищённые соединения на порту 443 (см. рис. Рисунок 2.3).

```
[root@server.eegashimova.net conf.d]#  
[root@server.eegashimova.net conf.d]# firewall-cmd --add-service=https  
success  
[root@server.eegashimova.net conf.d]# firewall-cmd --add-service=https --permanent  
success  
[root@server.eegashimova.net conf.d]# firewall-cmd --reload  
success  
[root@server.eegashimova.net conf.d]# systemctl restart httpd  
[root@server.eegashimova.net conf.d]# █
```

Рисунок 2.3: Разрешение HTTPS в firewalld и перезапуск HTTP-сервера

2.4 Проверка работы веб-сервера по HTTPS

На виртуальной машине client открываю в браузере адрес `www.eegashimova.net`. Настроенное правило перенаправления переводит соединение с HTTP на HTTPS. Поскольку для сервера используется самостоятельно подписанный сертификат, браузер не доверяет его издателю и выводит предупреждение `SEC_ERROR_UNKNOWN_ISSUER`. Перехожу к дополнительным параметрам и разрешаю использование данного сертификата для тестового веб-сервера (см. рис. Рисунок 2.4).

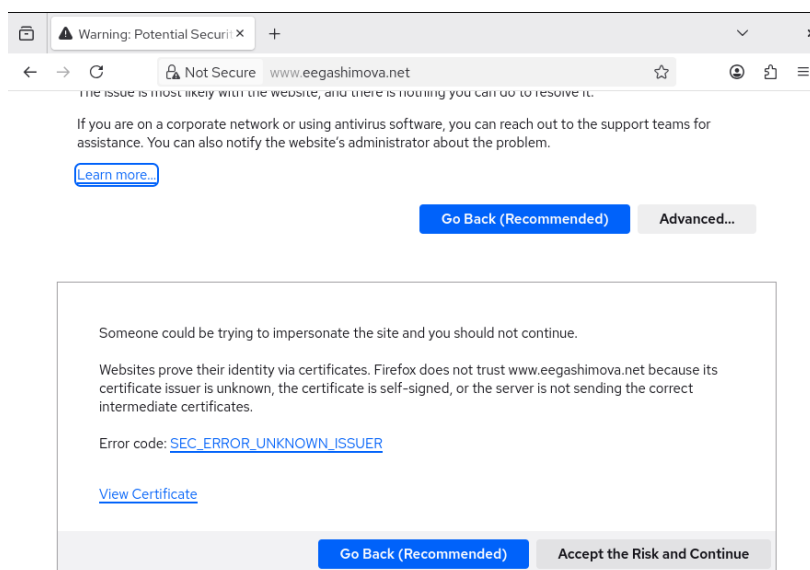


Рисунок 2.4: Предупреждение браузера при использовании самоподписанного сертификата

После добавления исключения браузер открывает страницу виртуального хоста `www.eegashimova.net` по защищённому соединению. В адресной строке отображается значок защищённого соединения, а содержимое созданной ранее страницы успешно загружается с веб-сервера (см. рис. Рисунок 2.5).

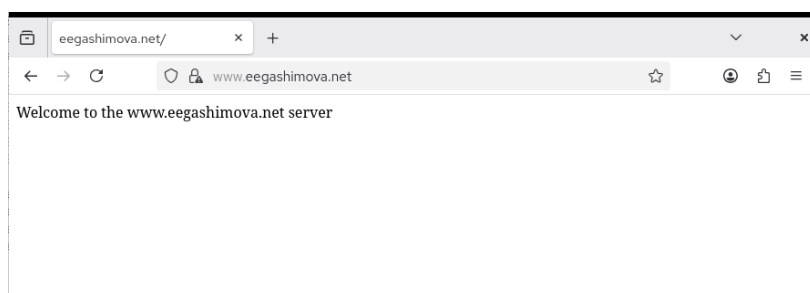


Рисунок 2.5: Проверка доступа к веб-серверу по протоколу HTTPS

Просматриваю сведения о сертификате в браузере. В поле страны указано RU, региона — Russia, города — Moscow. В качестве организации и подразделения указано eegashimova, общее имя сертификата — eegashimova.net, а адрес электронной почты — eegashimova@eegashimova.net. Поскольку сертификат яв-

ляется самоподписанным, данные субъекта и издателя совпадают (см. рис. Рисунок 2.6).

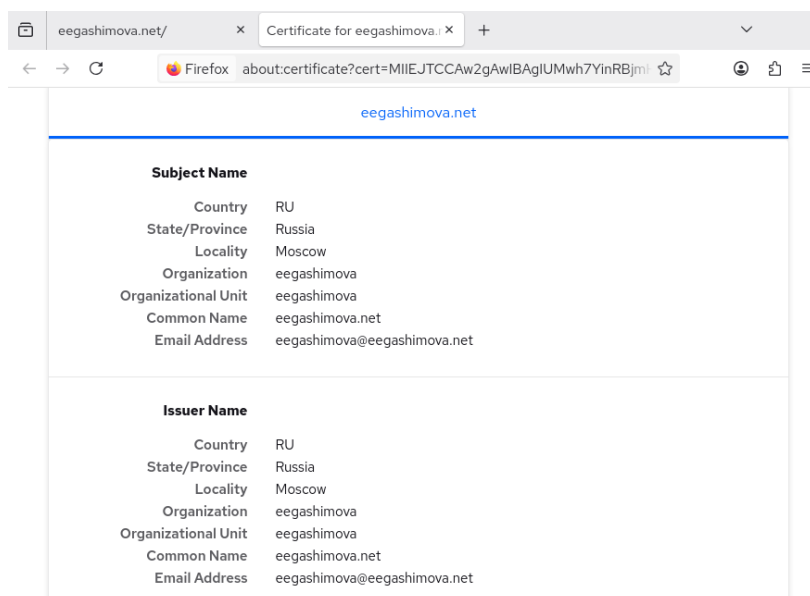


Рисунок 2.6: Просмотр данных самоподписанного сертификата

2.5 Настройка HTTP-сервера для работы с РНР

Для добавления поддержки РНР устанавливаю пакет `php` с помощью пакетного менеджера `dnf`. Вместе с основным пакетом устанавливаются необходимые зависимости и дополнительные компоненты, в том числе `php-cli`, `php-common`, `php-fpm`, `php-mbstring`, `php-opcache`, `php-pdo` и `php-xml`. Установка завершается сообщением `Complete!` (см. рис. Рисунок 2.7).

```

Total                                                                 11 MB/s | 8.2 MB   00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                : 1/1
  Installing                : php-common-8.3.33-1.el10_2.x86_64 1/10
  Installing                : php-cli-8.3.33-1.el10_2.x86_64 2/10
  Installing                : php-mbstring-8.3.33-1.el10_2.x86_64 3/10
  Installing                : php-pdo-8.3.33-1.el10_2.x86_64 4/10
  Installing                : php-xml-8.3.33-1.el10_2.x86_64 5/10
  Running scriptlet: nginxfilesystem-2:1.26.3-6.el10_2.7.noarch 6/10
  Installing                : nginxfilesystem-2:1.26.3-6.el10_2.7.noarch 6/10
  Installing                : php-fpm-8.3.33-1.el10_2.x86_64 7/10
  Running scriptlet: php-fpm-8.3.33-1.el10_2.x86_64 7/10
  Installing                : capstone-5.0.1-8.el10_2.x86_64 8/10
  Installing                : php-opcache-8.3.33-1.el10_2.x86_64 9/10
  Installing                : php-8.3.33-1.el10_2.x86_64 10/10
  Running scriptlet: php-8.3.33-1.el10_2.x86_64 10/10

Installed:
capstone-5.0.1-8.el10_2.x86_64      nginxfilesystem-2:1.26.3-6.el10_2.7.noarch  php-8.3.33-1.el10_2.x86_64
php-cli-8.3.33-1.el10_2.x86_64      php-common-8.3.33-1.el10_2.x86_64          php-fpm-8.3.33-1.el10_2.x86_64
php-mbstring-8.3.33-1.el10_2.x86_64  php-opcache-8.3.33-1.el10_2.x86_64        php-pdo-8.3.33-1.el10_2.x86_64
php-xml-8.3.33-1.el10_2.x86_64

Complete!
[root@server.eegashimova.net conf.d]#

```

Рисунок 2.7: Установка PHP и необходимых зависимостей

В каталоге веб-сайта `/var/www/html/www.eegashimova.net` создаю файл `index.php`. В файл помещаю вызов функции `phpinfo()`, которая выводит информацию о текущей конфигурации PHP, его версии, подключённых модулях и параметрах среды выполнения (см. рис. Рисунок 2.8).

```

<?php
    phpinfo();
?>

```

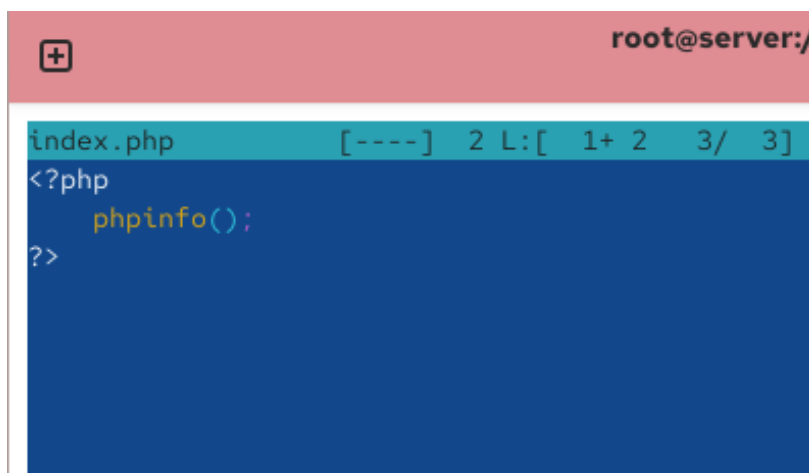


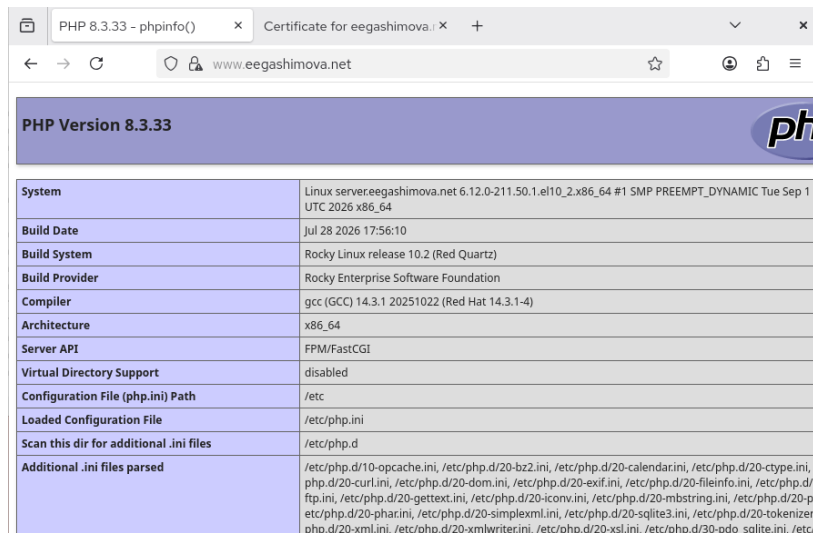
Рисунок 2.8: Создание файла `index.php` с вызовом `phpinfo`

После создания PHP-страницы изменяю владельца содержимого каталога `/var/www` на пользователя и группу `apache` командой `chown -R apache:apache /var/www/`. Далее восстанавливаю контексты безопасности SELinux для каталогов `/etc` и `/var/www` при помощи `restorecon -vR`. После выполнения настроек перезапускаю службу `httpd`, чтобы веб-сервер продолжил работу с обновлённой конфигурацией (см. рис. Рисунок 2.9).

```
[root@server.eegashimova.net www.eegashimova.net]#
[root@server.eegashimova.net www.eegashimova.net]# chown -R apache:apache /var/www/
[root@server.eegashimova.net www.eegashimova.net]# restorecon -vR /etc
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0
[root@server.eegashimova.net www.eegashimova.net]# restorecon -vR /var/www/
[root@server.eegashimova.net www.eegashimova.net]# systemctl restart httpd
[root@server.eegashimova.net www.eegashimova.net]#
```

Рисунок 2.9: Настройка прав, восстановление контекстов SELinux и перезапуск Apache

На виртуальной машине `client` повторно открываю адрес `www.eegashimova.net`. Вместо исходной HTML-страницы отображается результат выполнения функции `phpinfo()`. На странице указана установленная версия PHP 8.3.33, информация об операционной системе, архитектуре `x86_64`, Server API `FPM/FastCGI`, расположении файла `/etc/php.ini` и другие параметры PHP. Появление данной страницы подтверждает корректную обработку PHP-файлов веб-сервером (см. рис. Рисунок 2.10).



PHP Version 8.3.33	
System	Linux server.eegashimova.net 6.12.0-211.50.1.el10_2.x86_64 #1 SMP PREEMPT_DYNAMIC Tue Sep 11 10:00:00 UTC 2026 x86_64
Build Date	Jul 28 2026 17:56:10
Build System	Rocky Linux release 10.2 (Red Quartz)
Build Provider	Rocky Enterprise Software Foundation
Compiler	gcc (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4)
Architecture	x86_64
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype.ini, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/php.d/20-ftp.ini, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-mbstring.ini, /etc/php.d/20-pdo.ini, /etc/php.d/20-phar.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sqlite3.ini, /etc/php.d/20-tokenizer.ini, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlwriter.ini, /etc/php.d/20-xsl.ini, /etc/php.d/30-pdo_sqlite.ini, /etc/

Рисунок 2.10: Проверка выполнения PHP на веб-сервере

2.6 Внесение изменений в настройки внутреннего окружения виртуальной машины

Для сохранения выполненной конфигурации в проекте Vagrant копирую содержимое `/etc/httpd/conf.d` в каталог `/vagrant/provision/server/http/etc/httpd/conf.d`, а содержимое `/var/www/html` — в `/vagrant/provision/server/http/var/www/html`. При копировании подтверждаю замену существующих конфигурационных файлов.

Затем создаю в каталоге provisioning структуру `/vagrant/provision/server/http/etc/pki/tls` и `/vagrant/provision/server/http/etc/pki/tls/certs`. В эти каталоги копирую соответственно закрытый ключ `www.eegashimova.net.key` и сертификат `www.eegashimova.net.crt`. В результате конфигурация веб-сервера, содержимое сайта и криптографические файлы сохраняются в проекте Vagrant и могут быть использованы при повторном создании виртуальной машины (см. рис. Рисунок 2.11).

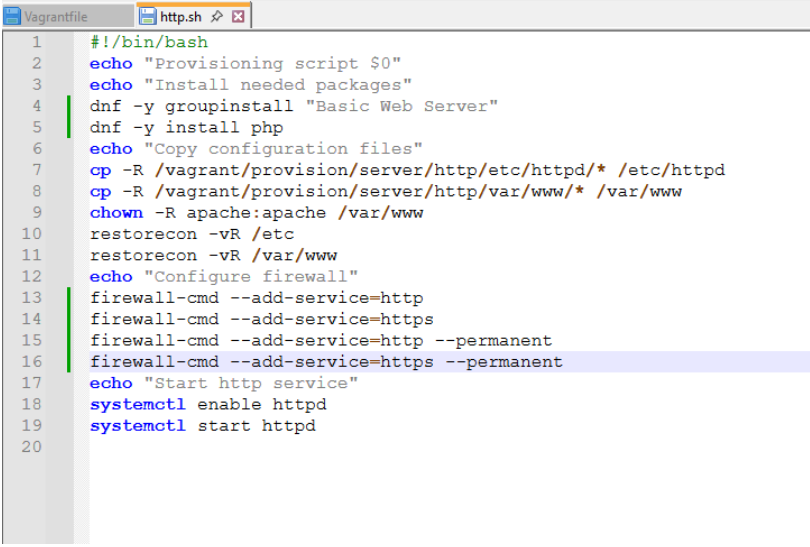
```
[root@server.eegashimova.net www.eegashimova.net]#
[root@server.eegashimova.net www.eegashimova.net]# cp -R /etc/httpd/conf.d/* /vagrant/provision/server/http/etc/httpd/conf.d/
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/autoindex.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/cgiid.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/manual.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/README'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/server.eegashimova.net.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/ssl.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/userdir.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/welcome.conf'? y
cp: overwrite '/vagrant/provision/server/http/etc/httpd/conf.d/www.eegashimova.net.conf'? y
[root@server.eegashimova.net www.eegashimova.net]# cp -R /var/www/html/* /vagrant/provision/server/http/var/www/html/
cp: overwrite '/vagrant/provision/server/http/var/www/html/server.eegashimova.net/index.html'? y
[root@server.eegashimova.net www.eegashimova.net]# mkdir -p /vagrant/provision/server/http/etc/pki/tls/private
[root@server.eegashimova.net www.eegashimova.net]# mkdir -p /vagrant/provision/server/http/etc/pki/tls/certs
[root@server.eegashimova.net www.eegashimova.net]# cp -R /etc/pki/tls/private/www.eegashimova.net.key /vagrant/provision/server/http/etc/pki/tls/private/
[root@server.eegashimova.net www.eegashimova.net]#
[root@server.eegashimova.net www.eegashimova.net]# cp -R /etc/pki/tls/certs/www.eegashimova.net.crt /vagrant/provision/server/http/etc/pki/tls/certs/
[root@server.eegashimova.net www.eegashimova.net]# █
```

Рисунок 2.11: Копирование конфигурации, веб-контента, ключа и сертификата в проект Vagrant

Редактирую скрипт `/vagrant/provision/server/http.sh`, отвечающий за автоматическую настройку HTTP-сервера. В существующий сценарий добавляю установку PHP командой `dnf -y install php`. Скрипт копирует сохранённые конфигурационные файлы и содержимое `/var/www`, устанавливает владельца `apache:apache`, восстанавливает контексты SELinux и настраивает межсетевой

экран.

Для межсетевого экрана добавляю разрешение сервисов http и https как в текущую, так и в постоянную конфигурацию firewalld. В заключительной части скрипта служба httpd добавляется в автозагрузку и запускается. В результате расширенная настройка HTTP-сервера, включая поддержку HTTPS и PHP, фиксируется в сценарии автоматического развёртывания виртуальной машины (см. рис. Рисунок 2.12).

A screenshot of a Vagrantfile editor window. The window has two tabs: 'Vagrantfile' and 'http.sh'. The 'http.sh' tab is active, showing a shell script with 20 lines of code. The script is a provisioning script for an HTTP server. It starts with a shebang line, followed by echo statements for logging. It then uses 'dnf' to install the 'Basic Web Server' group and the 'php' package. Next, it copies configuration files from a Vagrant provision directory to the local system, specifically for httpd and www. It then uses 'restorecon' to set SELinux permissions on the copied files. Finally, it configures the firewall to allow http and https traffic, enables the httpd service, and starts it.

```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y groupinstall "Basic Web Server"
5  dnf -y install php
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
8  cp -R /vagrant/provision/server/http/var/www/* /var/www
9  chown -R apache:apache /var/www
10 restorecon -vR /etc
11 restorecon -vR /var/www
12 echo "Configure firewall"
13 firewall-cmd --add-service=http
14 firewall-cmd --add-service=https
15 firewall-cmd --add-service=http --permanent
16 firewall-cmd --add-service=https --permanent
17 echo "Start http service"
18 systemctl enable httpd
19 systemctl start httpd
20
```

Рисунок 2.12: Скрипт автоматической настройки HTTP-сервера с поддержкой HTTPS и PHP

3 Вывод

В ходе лабораторной работы веб-сервер Apache настроен для работы по протоколу HTTPS. Сгенерированы закрытый ключ и самоподписанный сертификат, настроено перенаправление с HTTP на HTTPS и разрешён сервис https в firewalld.

Также установлена поддержка PHP и выполнена проверка работы с помощью функции `phpinfo()`. В завершение конфигурация сервера сохранена в проекте Vagrant, а скрипт `http.sh` дополнен настройками HTTPS и PHP.

4 Контрольные вопросы

1. В чём отличие HTTP от HTTPS?

HTTP передаёт данные без шифрования и обычно использует порт 80. HTTPS использует TLS для шифрования передаваемых данных и работает через порт 443.

2. Каким образом обеспечивается безопасность контента веб-сервера при работе через HTTPS?

HTTPS использует протокол TLS, который обеспечивает шифрование данных, проверку их целостности и подтверждение подлинности веб-сервера с помощью цифрового сертификата.

3. Что такое сертификационный центр? Приведите пример.

Сертификационный центр (CA) — организация, которая выпускает и подписывает цифровые сертификаты, подтверждая их подлинность. Пример сертификационного центра — Let's Encrypt.